

# FitBuddy – Detailed Project Design

## AI-Assisted Fitness Planning and Athlete Management System

This project design defines the functional, technical, interface, data, and security architecture of FitBuddy. The system is designed to help athletes create and follow personalized workout plans while allowing coaches or administrators to inspect athletes, plans, progress, and AI-assisted feedback revisions.

### 1. Project Overview

| Area           | Design Description                                                                                            |
|----------------|---------------------------------------------------------------------------------------------------------------|
| Project Name   | FitBuddy                                                                                                      |
| Primary Users  | Athletes, coaches, and administrators                                                                         |
| Core Purpose   | Personalized fitness planning, workout tracking, progress monitoring, and feedback management                 |
| Key Technology | Responsive web application with a structured frontend, backend services, database, and optional AI feedback I |
| Main Outcome   | A centralized platform for managing fitness activities and generating actionable workout guidance             |

### 2. System Architecture

FitBuddy follows a layered architecture. The presentation layer handles the user interface and responsive interactions. The application layer manages authentication, workout-plan logic, progress calculations, coach workflows, and AI feedback. The data layer stores users, athletes, plans, exercises, sessions, progress records, and feedback revisions.

| Layer        | Main Components                                                 | Responsibility                                              |
|--------------|-----------------------------------------------------------------|-------------------------------------------------------------|
| Presentation | Login, dashboards, forms, workout screens, progress views       | User interaction and visualization                          |
| Application  | Authentication, plan service, athlete service, feedback service | Business rules and workflows                                |
| AI/Analytics | Recommendation and feedback module                              | Generate or revise feedback based on available fitness data |
| Data         | Relational database / structured storage                        | Persistent application data                                 |
| Security     | Authorization, validation, secure sessions                      | Protect accounts and restrict access                        |

### 3. Functional Module Design

#### 3.1 Authentication & User Management

- User registration and login.
- Role-based access for athlete, coach, and administrator.
- Profile management and account settings.
- Session handling and logout.

#### 3.2 Athlete Module

- View personal dashboard and assigned workout plans.
- Record completed exercises, repetitions, duration, and notes.

- Track workout history and progress.
- Review coach or AI-generated feedback.

### **3.3 Workout Plan Module**

- Create structured plans by goal, fitness level, duration, and available equipment.
- Organize plans into days, exercises, sets, repetitions, rest periods, and notes.
- Allow plan updates while preserving useful history.
- Display current and completed plans clearly.

### **3.4 Coach/Admin Dashboard**

- View registered athletes.
- Inspect athlete profiles and workout plans.
- Review progress and submitted workout information.
- Create, edit, assign, and revise plans.
- Manage AI feedback revisions and administrative records.

### **3.5 AI Feedback Module**

- Accept relevant athlete/workout inputs.
- Generate structured feedback such as observations, improvement areas, and suggested next actions.
- Keep AI output separate from verified medical or clinical advice.
- Allow authorized human review before important plan changes.

### **3.6 Progress & Analytics**

- Summarize completed workouts and adherence.
- Display progress using simple charts and cards.
- Compare planned versus completed activity.
- Provide date-based filtering for historical analysis.

## 4. User Interface Design

| Screen            | Important UI Elements                                                          |
|-------------------|--------------------------------------------------------------------------------|
| Landing Page      | Project introduction, benefits, navigation, sign-in/register actions           |
| Login/Register    | Email/username, password, validation messages, account actions                 |
| Athlete Dashboard | Profile summary, today's workout, completion status, recent progress, feedback |
| Workout Detail    | Exercise name, sets, reps, rest time, instructions, completion controls        |
| Progress Page     | Workout history, completion statistics, trend charts, filters                  |
| Coach Dashboard   | Athlete list, search/filter, plan status, feedback review                      |
| Plan Builder      | Goal, duration, exercises, sets/reps, rest, notes, assignment controls         |
| Admin View        | User management, plan overview, feedback revision, system-level monitoring     |

## 5. Database Design

| Entity           | Key Fields                                         | Purpose                            |
|------------------|----------------------------------------------------|------------------------------------|
| User             | user_id, name, email, password_hash, role          | Authentication and role management |
| AthleteProfile   | athlete_id, user_id, age, fitness_level, goals     | Fitness profile information        |
| WorkoutPlan      | plan_id, title, goal, duration, created_by, status | Stores workout plans               |
| PlanExercise     | item_id, plan_id, exercise_id, sets, reps, rest    | Exercises inside a plan            |
| Exercise         | exercise_id, name, category, instructions          | Reusable exercise catalogue        |
| WorkoutSession   | session_id, athlete_id, plan_id, date, status      | Records completed/planned sessions |
| ProgressRecord   | record_id, athlete_id, metric, value, date         | Stores progress measurements       |
| Feedback         | feedback_id, athlete_id, source, content, status   | Coach/AI feedback                  |
| FeedbackRevision | revision_id, feedback_id, editor, version, content | Tracks feedback changes            |

## 6. Core Data Flow

1. User registers or logs in → authentication service validates credentials.
2. Athlete completes or updates profile → profile service stores fitness goals and preferences.
3. Workout plan is created/assigned → plan service stores exercises and schedule.
4. Athlete records a workout → session and progress records are updated.
5. Feedback service analyzes permitted inputs → structured feedback is produced.
6. Coach/admin reviews or revises feedback → revision history is preserved.
7. Dashboard retrieves current data → user sees plan status, history, and progress.

## 7. API / Service Design

| Endpoint / Service       | Method         | Purpose                              |
|--------------------------|----------------|--------------------------------------|
| /auth/register           | POST           | Create a user account                |
| /auth/login              | POST           | Authenticate a user                  |
| /athletes/{id}           | GET/PUT        | Read or update athlete profile       |
| /plans                   | GET/POST       | List or create workout plans         |
| /plans/{id}              | GET/PUT/DELETE | Manage one workout plan              |
| /sessions                | POST/GET       | Record and retrieve workout sessions |
| /progress                | GET/POST       | Read or add progress records         |
| /feedback                | GET/POST       | Create and retrieve feedback         |
| /feedback/{id}/revisions | GET/POST       | Manage feedback revision history     |

## 8. Security Design

- Use hashed passwords rather than storing plain-text passwords.
- Apply role-based authorization so users can access only permitted features.
- Validate and sanitize user input on both client and server sides.
- Use HTTPS in production and secure session/token handling.
- Restrict sensitive athlete data to authorized users.
- Maintain audit information for important plan and feedback changes.
- Apply rate limiting and appropriate error handling to authentication and API endpoints.
- Avoid presenting AI-generated output as medical diagnosis or guaranteed health advice.

## 9. Non-Functional Requirements

| Requirement     | Design Target                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------------|
| Performance     | Common dashboard/API actions should respond quickly under normal load.                           |
| Usability       | Simple navigation, readable cards, clear status indicators, mobile-friendly layout.              |
| Scalability     | Modular services and indexed database fields should support growth in users and workout records. |
| Reliability     | Validation, error handling, backups, and transaction-safe updates.                               |
| Maintainability | Reusable components, clear naming, modular services, documentation, and version control.         |
| Accessibility   | Keyboard-friendly controls, readable contrast, labels, and meaningful feedback messages.         |

## 10. Testing Strategy

- Unit testing: validate individual functions, calculations, form validation, and service methods.
- Integration testing: verify communication between authentication, plan, session, progress, and feedback services.
- UI testing: verify navigation, responsive layouts, forms, dashboard cards, and workout completion flows.
- Security testing: test authorization boundaries, invalid inputs, session handling, and common web vulnerabilities.
- User acceptance testing: verify that athlete and coach workflows match project requirements.
- Performance testing: measure response time and behavior as the number of users and records increases.

## 11. Deployment Design

A practical deployment can separate the frontend, backend/API, database, and optional AI service. The frontend serves the responsive application, the backend exposes authenticated APIs, the database persists application data, and the AI component handles permitted feedback-generation requests. Environment variables should be used for secrets and service configuration.

## 12. Future Enhancements

- Wearable-device integration for activity and workout metrics.
- Exercise video demonstrations and form guidance.
- Push notifications for workout schedules and reminders.

- Advanced progress analytics and customizable dashboards.
- Coach-to-athlete messaging and plan approval workflows.
- Personalized recommendation models using validated historical data.
- Offline-friendly mobile experience or dedicated mobile application.

## **13. Conclusion**

The FitBuddy project design provides a structured foundation for a fitness-management platform that combines workout planning, athlete tracking, coach administration, progress analytics, and AI-assisted feedback. The modular architecture allows the project to start with essential fitness workflows and expand later without redesigning the entire system.